BUILD BRIEF

UTM Sales Attribution & Profitability Report Generator

Antigravity Implementation Specification

Primary source of truth for the project. Accuracy is more important than speed.

1. Project Objective

Build a robust, repeatable automation tool that takes raw marketing and sales exports and generates a complete UTM-based sales attribution and campaign profitability report.

- Leads data ingestion
- Sales/buyers data ingestion
- Multiple Meta Ads account spend ingestion
- File inspection, column detection and mapping
- Data cleaning and normalization
- Lead deduplication
- Sales processing and attribution
- Meta spend attribution
- Free vs Paid webinar analysis
- Old vs New lead analysis
- Revenue, spend, profit, ROAS, ROI, CPL, CAC and conversion calculations
- Campaign / Ad Set / Creative reporting
- Waste detection
- Verification
- Final multi-sheet Excel generation

2. Source-of-Truth Rule

This document is the primary business specification. Do not invent business rules, silently change formulas, use standard UTM conventions, simplify attribution, silently guess required columns, or ignore verification failures.

If a business decision is unresolved, clearly report it rather than silently inventing an answer.

3. Current Project Condition

No real client datasets are currently available. There is no Leads file, Sales file, Meta Ads Spend file, or historical manual report.

For development and testing, generate realistic synthetic/mock datasets. The architecture must support real CSV/XLSX files later without changing core business logic. Never claim real-data validation until real datasets are actually provided.

4. Existing Components / Skills

Before writing new logic, inspect whether these existing skills/components are actually available:

```
utm-sales-attribution-report
roas-report
```

If they exist, inspect and reuse valid logic where appropriate. If they do not exist, implement the required functionality from this brief. Never claim an existing component exists unless it is actually available.

5. Technology Stack

• Python 3.11+

• Pandas and NumPy where required

• OpenPyXL for Excel

• Pytest for testing

• JSON configuration files

• Streamlit for Phase 2 UI

Do not use React/Next.js for the core automation engine. Do not add a database or cloud storage unless explicitly required later.

6. Input Files

6.1 Leads

Support CSV and XLSX. Required logical fields: Email, Registration Date / created_at, utm_source, utm_medium, utm_content, Webinar Type. Preferred/optional: Name, Phone, utm_campaign, Webinar/Batch Name, Registration Fee Paid.

6.2 Sales

Support CSV and XLSX. Required: Email and Sale/Order Date. Strongly preferred: Order Amount. Optional: Product/Offer Name, UTM columns, Payment Status.

If payment status exists, include only successful/captured/completed sales. Exclude failed, pending and refunded sales, and log excluded rows separately.

6.3 Meta Ads Spend

Support multiple CSV/XLSX files, each representing one Meta Ads account. Required: Campaign Name, Ad Set Name, Ad Name, Amount Spent, Day. Preferred: Impressions, Clicks, Results. Account name should come from the filename or a user-entered account label.

7. Critical UTM Convention

This project does NOT use standard industry UTM meanings.

```
utm_source = Campaign Name
utm_medium = Ad Set Name
utm_campaign = Placement
utm_content = Ad Creative / Ad Name
```

utm_campaign is placement, NOT campaign name, and is NOT used for attribution.

8. Settings

• Report Name / Client Name

• Start Date and End Date (inclusive)

• New vs Old Lead Cutoff Date

• Fallback Price Per Sale

• Zero-ROI Waste Threshold (default ₹5,000)

• Currency

• FX Rate where required

9. File Inspection & Column Mapping

Inspect every input for filename, type, sheet names, columns, row counts, date ranges, spend, distinct campaigns/ad sets/ads, distinct UTM values, missing required fields and duplicates. Detect spend outside the reporting window, campaigns across multiple ad accounts, empty files and missing required columns.

Use configurable aliases in config/aliases.json. If a required column cannot be confidently detected, request manual mapping; never silently guess.

10. Cleaning & Normalization

• Fix common mojibake/encoding corruption where appropriate.

• Trim and collapse spaces; match campaign/ad set/ad names case-insensitively while preserving display casing.

• Support campaign unification mapping.

• Normalize emails to lowercase and trim whitespace.

• Normalize phones by removing +91, spaces, dashes, brackets and other formatting differences.

11. Lead & Sales Deduplication

For duplicate lead emails, select the most recent registration that contains usable UTM data; do not simply select the latest row. Keep audit information. Treat each valid sales row as a separate sale unless explicitly instructed otherwise. Report duplicate sales emails. Exclude failed/refunded/pending orders according to payment status.

12. Invalid / Unusable UTM Values

```
blank
--
null
{{campaign.name}}
{{adset.name}}
{{ad.name}}
fb
facebook
paid
instagram
google
ig
```

Purely numeric utm_content is also unusable. Store the configurable list in config/sentinels.json.

13. Sales Attribution Priority

```
1. Sales Sheet UTM
2. Leads DB by Email
3. Leads DB by Phone
4. Unattributed
```

Never fabricate attribution.

Attribution Source and Match Level are separate. Match Level must be one of: Ad Level, Adset Level, Campaign Level, Unattributed.

14. Meta Spend Attribution

Tier 1 — Ad Level

```
Campaign + Ad Set + Ad
Attributed Spend Per Sale = Exact Ad Window Spend / Number of Sales attributed to that exact Ad
```

Tier 2 — Ad Set Level

```
If exact Ad is not found:
Attributed Spend Per Sale = Ad Set Window Spend / Ad Set Level Sales Count
```

Tier 3 — Campaign Level

```
If Ad Set is not found:
Attributed Spend Per Sale = Campaign Window Spend / Campaign Level Sales Count
```

Tier 0 — Unattributed

```
If campaign cannot be matched:
Attributed Spend = 0
```

NEVER assign full spend to every sale. Spend must be divided proportionally at every tier.

Hard invariant: Total Attributed Spend <= Total Windowed Meta Spend. If this fails, the report is INVALID and must not be delivered as successful.

15. Old vs New Leads

```
New = registered on or after cutoff date
Old = registered before cutoff date
```

Old sales remain in All Sales and are flagged Old, but are excluded from headline New Lead ROAS. Show both ROAS (New Leads Only) and ROAS (All Sales).

16. Free vs Paid Webinar Logic

Free Webinar

```
Revenue = Backend Sale Amount
Registration Revenue = 0
```

Paid Webinar

```
Revenue = Registration Fee + Backend Sale Amount
```

Registration fee revenue counts for every paid registration in the reporting window, even when the registrant does not later purchase the backend product. Registration fee attribution uses the lead's own UTM data.

17. Free vs Paid Reporting

The Free vs Paid Funnel must separately show Free, Paid and Blended metrics: Spend, Leads, CPL, Registration Revenue, Sales, Conversion Rate, Backend Revenue, Total Revenue, Profit, ROAS and CAC. Never show only blended CPL or ROAS.

18. Currency / FX

Default currency: INR. If an ad account uses USD/AED/etc., support a user-supplied FX rate and record source currency, target currency, FX rate and rate date in Settings & Run Log. Never silently assume a conversion rate.

19. Metric Definitions

```
Revenue = Order Amount + Registration Fee where applicable
Spend = Attributed Spend
Profit = Revenue - Spend
ROAS = Revenue / Spend
ROI % = Profit / Spend × 100
CPL = Spend / Leads
CAC = Spend / Sales
Conversion Rate % = Sales / Leads × 100
Profitable = YES if Profit > 0, otherwise NO
```

If a denominator is zero, return blank. Never output #DIV/0!, Infinity, NaN or fake zero values.

20. Attribution Mix & Waste

Show Ad Level %, Adset Level %, Campaign Level % and Unattributed %. If Unattributed > 10%, show a visible UTM tracking warning.

Zero-ROI Waste Report: at Campaign, Ad Set and Ad Creative level, find nodes where Spend >= ₹5,000 and Sales = 0. Report level, node name, ad account, spend, sales, revenue and reason.

21. Final Excel Workbook

Generate one .xlsx workbook with EXACTLY these 12 sheets in this order:

• 1. ⚙ Settings & Run Log

• 2. □ All Sales (Attributed)

• 3. □ Campaign Summary

• 4. □ Ad Set Summary

• 5. □ Ad Creative Summary

• 6. □ Ad Account Comparison

• 7. □ Free vs Paid Funnel

• 8. □ Zero-ROI Waste Report

• 9. □ Unattributed Sales

• 10. □ Old Leads (Separate)

• 11. □ Spend Reference

• 12. □ Verification

22. All Sales Sheet

One row per valid sale. Required columns:

> Sale ID | Email | Name | Sale Date | Registration Date | Webinar Type (Free/Paid) | Webinar/Batch | Product | Order Amount | Registration Fee | Total Revenue | Campaign | Ad Set | Ad Creative | Ad Account | Attribution Source | Match Level | Attributed Spend | Profit | ROAS | New/Old Lead | Notes

Every sale must clearly show attribution source, match level and attributed spend.

23. Summary Sheets

Campaign Summary, Ad Set Summary and Ad Creative Summary must contain: Node Name, Ad Account, Spend, Leads, CPL, Sales, Conversion Rate %, Revenue, Profit, ROAS, ROI %, CAC and Profitable? (YES/NO). Campaign Summary should be sorted by Sales descending.

24. Supporting Sheets

• Ad Account Comparison: Ad Account, Spend, Sales, Revenue, Profit, ROAS, CAC.

• Unattributed Sales: every unattributed sale, attempted methods and reason.

• Old Leads: all buyers registered before cutoff, flagged Old.

• Spend Reference: windowed raw Meta rows including Ad Account, Day, Campaign, Ad Set, Ad, Spend, Impressions, Clicks and Results.

• Settings & Run Log: client/report, dates, cutoff, fallback price, threshold, currency, FX, input filenames, row counts, exclusions, timestamp, spend window, attribution summary, assumptions and processing notes.

25. Excel Formatting

• Bold headers and freeze top row.

• Auto-fit columns.

• INR and Indian digit grouping.

• Percentage formatting and ROAS such as 3.4x.

• Bold totals.

• Green = profitable; Red = loss; Yellow = flagged/old/warning.

• Prioritize accuracy and readability over decoration.

26. Verification Engine — 9 Mandatory Checks

- 1. All Sales row count = Input Sales row count after refund exclusions; exclusions logged separately.
- 2. Campaign Sales = Ad Set Sales = Ad Creative Sales = Total Sales.
- 3. Campaign Revenue = Ad Set Revenue = Ad Creative Revenue.
- 4. Total Attributed Spend <= Total Windowed Meta Spend.
- 5. Summary spend totals reconcile with All Sales attributed spend.
- 6. Report duplicate sales-email count; flag if > 0.
- 7. Report Unattributed Count and %; warn if > 10%.
- 8. Every lead in the reporting window is counted exactly once in funnel summary.
- 9. Registration fee revenue equals qualifying registration fees in the Leads sheet for the reporting window.

If any mandatory check fails, do not report successful generation. Show REPORT INVALID with failed check, expected, actual, difference and reason. If Excel formulas are used, independently recalculate in Python and cross-check.

27. Testing

Because real data is unavailable, create synthetic datasets covering at least:

- Exact Ad match
- Ad missing → Ad Set fallback
- Ad Set missing → Campaign fallback
- Campaign missing → Unattributed
- Sales UTM
- Email fallback
- Phone fallback
- Invalid UTM
- Duplicate leads
- Duplicate sales
- Old lead
- New lead
- Free webinar
- Paid webinar
- Multiple Meta accounts
- Zero sales
- Zero spend
- Empty Meta account file
- Spend outside reporting window
- Spend invariant violation
- Missing optional columns
- Missing required columns
- Refund/failed payment exclusion
- Campaign spelling variations
- FX conversion
- Paid registration without backend purchase

28. Pytest Requirements

Test ingestion, inspection, mapping, normalization, matching, deduplication, sales processing, UTM sentinel detection, attribution hierarchy, spend allocation, old/new classification, Free/Paid revenue, registration revenue, metrics, verification and workbook generation. Run pytest and require all tests to pass before Phase 1 is complete.

29. Recommended Project Structure

```
utm-sales-attribution-generator/
├── main.py
├── requirements.txt
├── README.md
├── .gitignore
├── config/
│ ├── aliases.json
│ ├── sentinels.json
│ └── settings.json
├── src/
│ ├── ingestion.py
│ ├── inspection.py
│ ├── normalization.py
│ ├── leads.py
│ ├── sales.py
│ ├── attribution.py
│ ├── spend.py
│ ├── funnel.py
│ ├── metrics.py
│ ├── summaries.py
│ ├── workbook.py
│ ├── verification.py
│ └── logger.py
├── tests/
├── input/
├── output/
└── app/
└── streamlit_app.py
```

30. Phase 1 — Core Automation Engine

```
Ingest
↓
Inspect
↓
Clean
↓
Normalize
↓
Deduplicate
↓
Attribute Sales
↓
Attribute Spend
↓
Calculate Metrics
↓
Build Excel
↓
Verify
↓
Deliver
```

Phase 1 must be runnable from the command line. Configuration should control the run. No manual code edits should be required to process a new client dataset.

31. Phase 2 — Streamlit UI

Only after Phase 1 is fully tested and approved. Provide Leads upload, Sales upload, multiple Meta spend uploads, settings, manual column mapping when required, Generate Report, processing progress, verification result and

Excel download.

The Streamlit UI MUST call the same tested Phase 1 engine. Do not duplicate business logic inside Streamlit. No login, database or unnecessary cloud storage.

32. Data Privacy

Phase 2 should require no login, database or cloud storage; process files in memory where practical and discard uploaded files after processing. The system is intended for local/internal use.

33. Open Business Decisions

• Single-client or multi-client per run — current assumption: one client per run unless changed later.

• Attribution window behavior — Sales Sheet UTM wins if present; otherwise use lead attribution. This behaves as last-touch with first-touch fallback.

• Sales outside reporting window — e.g. lead registers during window but purchases three weeks later. Final business rule is not defined; do not invent one.

• Refund handling — refunded/failed orders are excluded; any separate net-of-refunds requirement must be explicitly introduced.

• Paid webinar registration fees — cash collected counts as revenue, including registrations without backend purchases.

• Phase 2 hosting — not defined; build for local execution first.

34. README Requirements

Document purpose, installation, Python version, dependencies, execution, input files, supported formats, column mapping, UTM convention, sentinel values, attribution priority, spend allocation, Free/Paid logic, Old/New logic, metrics, verification, workbook sheets, configuration, testing, Streamlit usage and troubleshooting.

35. Development Rule

```
WRITE → RUN → TEST → INSPECT → FIX → TEST AGAIN → VERIFY → CONTINUE
```

Never generate the entire project and assume it works.

36. Prohibited Mistakes

• Do not use standard UTM meanings.

• Do not treat utm_campaign as campaign.

• Do not silently guess required columns.

• Do not fabricate attribution.

• Do not assign full spend to every sale.

• Do not inflate spend, revenue or ROAS.

• Do not hide unattributed sales.

• Do not delete old sales.

• Do not blend Free and Paid without separate reporting.

• Do not count registration fees only for backend buyers.

• Do not use non-Meta spend as Meta spend.

• Do not divide by zero.

• Do not silently ignore failed verification.

• Do not claim real-data validation without real data.

• Do not duplicate business logic in Streamlit.

• Do not start Phase 2 before Phase 1 is validated.

• Do not add unnecessary database, authentication or cloud services.

37. Phase 1 Acceptance Criteria

• End-to-end pipeline works.

• CSV and XLSX supported.

• Multiple Meta accounts supported.

• Automatic and manual column mapping work.

• UTM convention and sentinel configuration work.

• Normalization, campaign unification and deduplication work.

• Sales attribution hierarchy works.

• All three spend tiers and proportional allocation work.

• Spend invariant is enforced.

• Old/New and Free/Paid logic work.

• FX handling and all metrics work.

• Attribution mix and waste reporting work.

• All 12 Excel sheets are generated in exact order.

• All 9 verification checks are implemented.

• Verification failure blocks successful delivery.

• Synthetic datasets and Pytest suite exist and pass.

• README is complete.

Real-data validation remains pending until real datasets are provided.

38. Final Execution Order

```
PHASE 0
Read brief → Inspect workspace → Inspect skills → Report existing/missing components → Identify unresolved decisions

PHASE 1
Project structure → Configuration → Ingestion → Inspection → Normalization → Lead processing
→ Sales processing → Attribution → Spend allocation → Free/Paid → Old/New
→ Metrics → Summaries → Excel → Verification → Synthetic data → Tests → Fix → Verify → Sign-off

PHASE 2
ONLY after Phase 1 approval
→ Streamlit UI → Connect tested engine → Test uploads/settings/report/verification/download

FINAL QA
→ Documentation → Acceptance checklist → Final delivery
```

39. First Action for Antigravity

DO NOT immediately start coding.

• Read this entire Build Brief.

• Inspect the workspace.

• Check whether utm-sales-attribution-report exists.

• Check whether roas-report exists.

• Inspect reusable implementations if available.

• Report what exists and what does not.

• Identify unresolved business decisions.

- Confirm that no real datasets are currently available.
- Create the Phase 1 implementation plan.
- Then begin Phase 1.

Do not start Phase 2 until Phase 1 has been fully tested and approved. Accuracy beats speed. Never claim completion without actual tests and verification.